

悪いマネージャーだった話

AI を自律的に動かす開発パイプライン

個人 OSS kobaamd の開発実験から

なぜ Markdown Editor を作るのか

個人 OSS kobaamd の動機 - Markdown が欲しかった理由 / AI 駆動を本気で試したかった理由

① Markdown Editor が欲しかった

AI 時代、Markdown は事実上の標準フォーマット

でも既存エディタは...

- ・ 重い (IDE 系)
- ・ クラウド前提 (Notion / Obsidian Sync)
- ・ Mac native 体験が薄い

→ 自分が一番使いたい道具がないなら作る

② AI 駆動を本気で試したかった

EM 歴は長いけど、モダン Web ・ Mac native は素人

「エンジニアじゃない自分でも、
AI に頼れば作れるのか？」

実験場としてちょうどよい OSS 規模

→ kobaamd の開発は AI 任せで進める

このLTのゴール：個人OSSで得た学びを、皆さんの業務に持ち込めるように

題材：kobaamd の規模感

2026-04-19 開始 → 21 日間で

21

日間

167

コミット

≈100

マージ PR

[画像 / 図 挿入領域]

kobaamd UI スクショ（後差し替え）

▶ 数字は当日直前に再取得

最初の作戦：3つのAIを役割分担

それぞれの特徴を見て、得意領域に当てはめた



Gemini

特徴

- ・ マルチモーダル
- ・ 大量入力に強い
- ・ 検索ベースの最新情報

→ だから この役割

リサーチ・調査



Codex CLI

特徴

- ・ コーディング特化
- ・ 単価が安い
- ・ ChatGPT Plus 認証で動く

→ だから この役割

実装の主力



Claude

特徴

- ・ 設計判断・推論力が高い
- ・ コードレビューが強い
- ・ 単価は高め

→ だから この役割

オーケストレーター

ところが Claude が、全部やろうとする

★悪いマネージャー①

「あれ？最近 Codex に頼んでる雰囲気がないな...」

「Claude さん、実装どうしてる？」

→ 「私が書きました！」

よくあるアンチパターン

「自分がやった方が早い」と思っている新任マネージャー

- ・ 設計判断と実装が混ざって品質ダウン
- ・ 単価の高いモデルが、本来は安いモデルでよい仕事までやる（コスト悪化）

役割を CLAUDE.md に明文化した

コードと同じくレビュー可能な場所にルールを置く

[画像 / 図 挿入領域]

CLAUDE.md の抜粋 (コードブロック)

効果メモ

[後で具体的に書く]

次の壁：人間が張り付かないと進まない

「これ、自分がボトルネックでは？」

あるあるな 1 日

1 Claude 「次どうしますか？」で作業が中断

2 PR が 3 つ溜まってる、レビューが追いつかない

3 通知でチラ見 → 5 分でも作業に戻れない

4 「あと一周だけ」で午前 2 時まで止まらない

5 テスト失敗の原因切り分けは結局自分

6 曖昧質問への待機状態が積み重なる

Cursor / Buddy / Claude Code、すべて『対話型』。同じ問題が起きる

全部任せた：自律パイプライン

13

SubAgent

28

Slash Command

3

定期実行バンドル

197

ステータス遷移

560

Linear API 書込

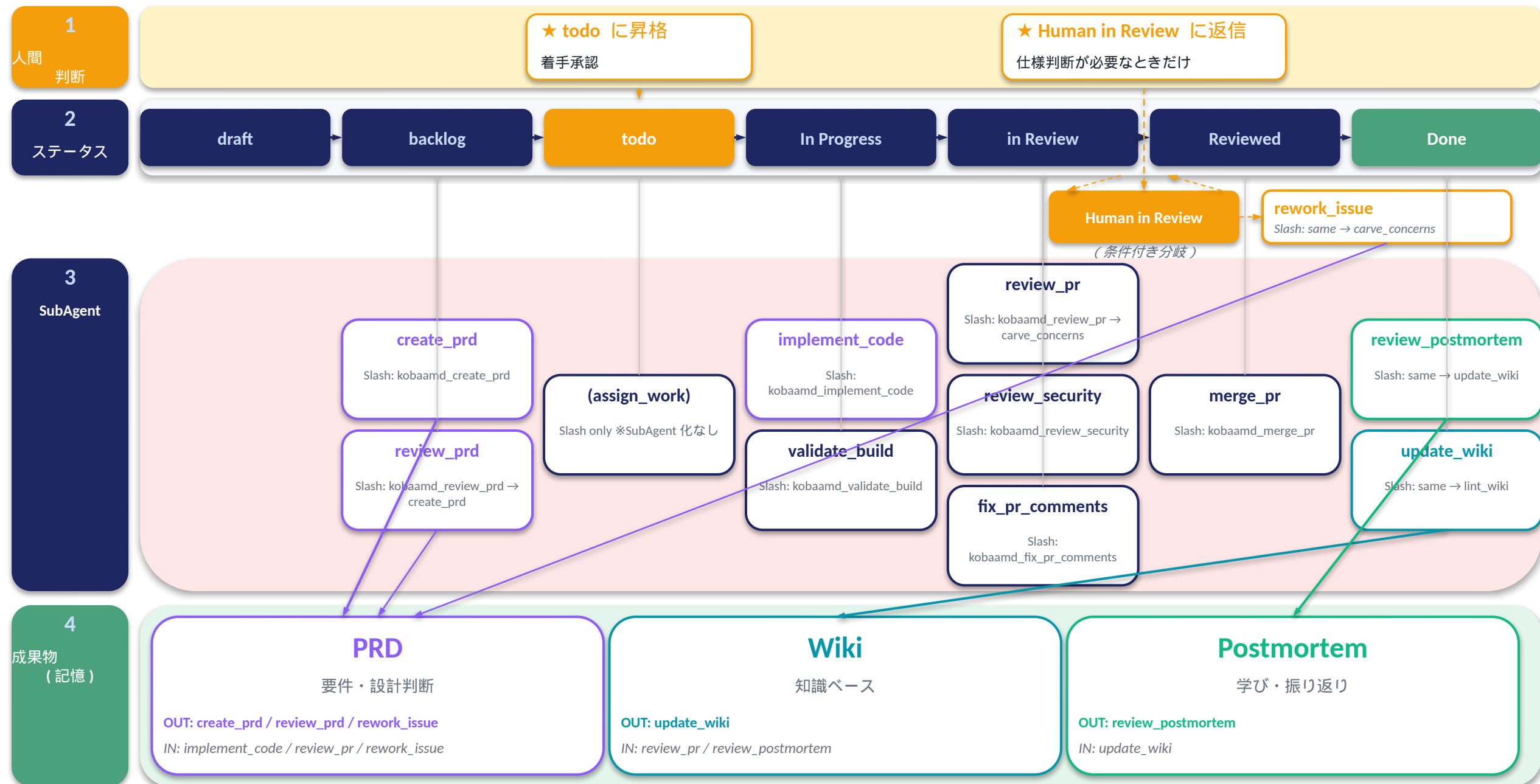
31+

Done 済 issue

人間がやるのは2つだけ：①投げる ②優先順位

▶ 数字は当日直前に再取得（Done 件数も Linear UI で目視）

ステータスフロー：通常は Human in Review を通らずにマージまで進む



でも、当然詰まる（実例）

状態ループ

KMD-145 : In Progress ↔ Todo を5回往復

コメントスルー

KMD-119 : Human in Review で72h+ 停滞

トークン枯渇

想定外の途中停止 → 人間が起こす

デグレ気付かず

ビルド通れば「ヨシ」、UI が壊れていても気付けない

→ 全部、人間チームでも起きる問題

仕組みを " 育てる " フェーズ + 限界

[Wiki 修正 / token 最適化 / ADR/PRD/Wiki に残す]

★ それでも限界：E2E の UX 評価は AI にできない！

ここで気付いた

実は私も、悪いマネージャーだった（★悪いマネージャー②）

01

指示が曖昧だと
出力も曖昧

02

手順を渡さないと
逐次質問される

03

AI には『積み重ね
の記憶』がない

対処：仕事の手順を Slash Command / SubAgent（md）に落とした

伝えたい3つのこと

01

AI を道具と思わず
人だと仮定して
仕組みを作ろう

02

AI が止まったら
仕組みの責任。
直そう

03

急に振りすぎない
優先順位をつけて
渡そう

「ほぼ人間を扱うみたいになってます」

まとめ

AI マネジメントは、
人間マネジメントの再発明だった

次へのチャレンジ

Token 最適化のさらなる挑戦

Sonnet / Haiku など軽量モデルでも回せる仕組み化 / 得意領域別の LLM 併用

E2E の UX 検証を Gemini あたりに任せる

「動く」と「使える」のギャップを自動化で埋める

「人間が判断すべきか？」を疑い続ける

残った人間ゲートをさらに減らせないか